

Secondary clustering occurs for any algorithm that generates the same sequence of probing for every key.

What we need are probe sequences that differ for each key instead of being the same for every key. Then numbers with different keys that hash to the same index will use different probe sequences.

The double hashing approach *hashes the key a second time, using a different method, and uses the result as the step size*. For a given key, the step size remains constant throughout a probe, but it's different for different keys. As long as the step size is not a multiple of the array size, it will eventually visit all the cells. That's one reason why prime numbers are good for the array size; they make it easier to avoid getting a step size that evenly divides the size of the array.

Experience has shown that this secondary hash function must have certain characteristics:

- It must not be the same as the primary hash function.
- It must never output a 0 (otherwise, there would be no step; every probe would land on the same cell).

Experts have discovered that functions of the following form work well:

```
stepSize = constant - (key % constant)
```

where `constant` is prime and smaller than the array size. For example,

```
stepSize = 5 - (key % 5)
```

The HashTableOpenAddressing Visualization tool uses this approach for its double hashing probe. Different keys may hash to the same index, but they will (most likely) generate different step sizes. With this algorithm and the `constant = 5`, the step sizes are all in the range 1 to 5. Two examples are shown in [Figure 11-16](#). The first search for key 4,678 starts at cell 78. The secondary hash function determines that the step size will be three for that key. After probing three filled cells, the function finds the desired key on the fourth step. The second search is for key 178 and starts at the same cell. For this key, however, the step size is determined to be four. After probing two filled cells, the function finds an empty cell on the third step.

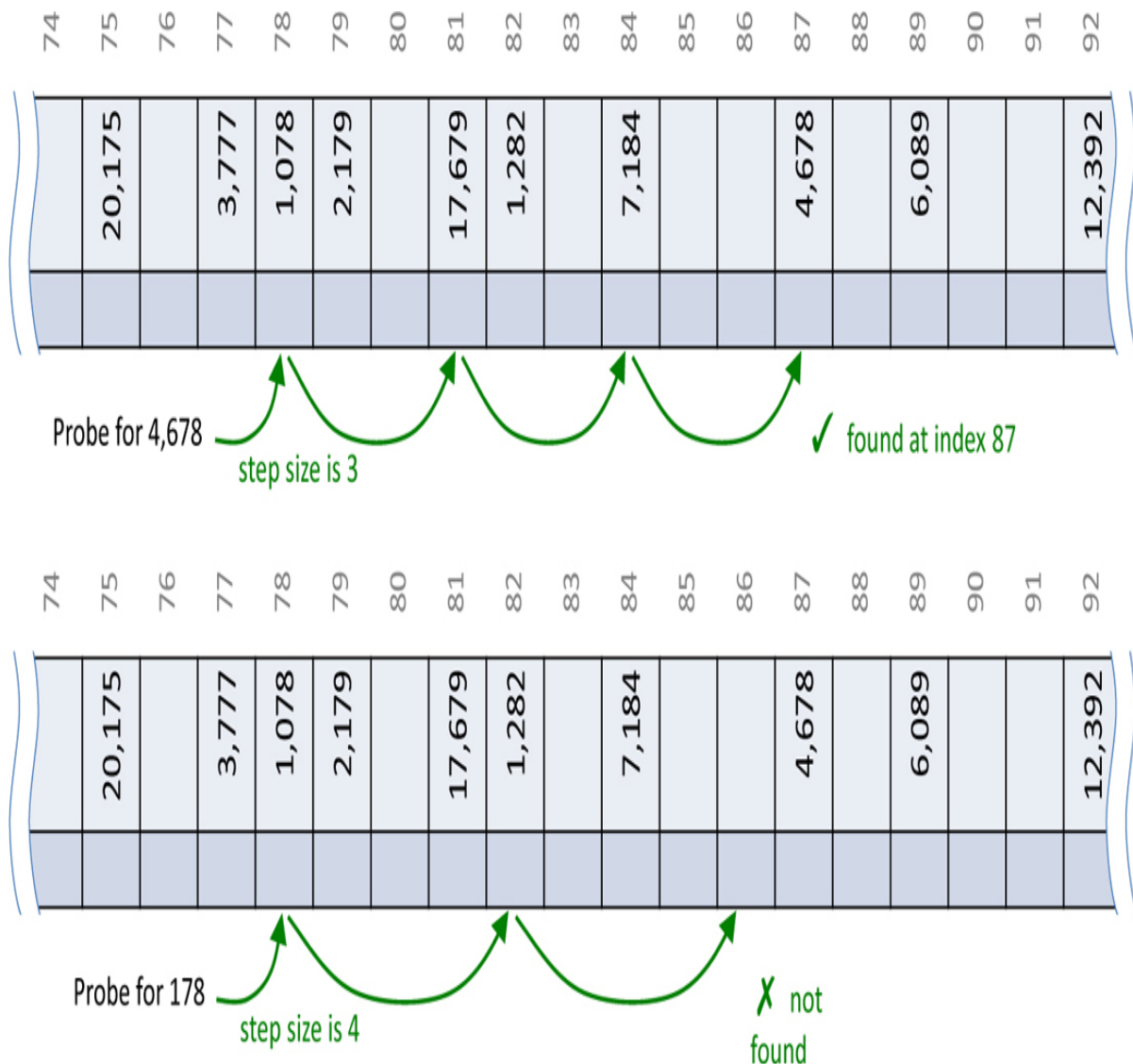


Figure 11-16 *Double hashing*

Implementing double hashing is only slightly more complicated than linear or quadratic probing. You still need a generator that loops over the possible indices, but this time you need to apply the secondary hash function to get the step size. That step should be less than or equal to a prime number below the size of the array. [Listing 11-10](#) shows an implementation for that generator.

Listing 11-10 *The `doubleHashProbe()` Generator for Open Addressing*

```
def doubleHashProbe(          # Generator to determine probe interval
    start, key, size):        # from a secondary hash of the key
```

```

yield start % size          # Yield the first cell index
step = doubleHashStep(key, size) # Get the step size for this key
for i in range(1, size): # Loop over all remaining cells using
    yield (start + i * step) % size # step from second hash of key

def doubleHashStep(key, size): # Determine step size for a given key
    prime = primeBelow(size) # Find largest prime below array size
    return prime - (          # Step size is based on second hash and
        simpleHash(key) % prime) # is in range [1, prime]

def primeBelow(n):
    # Find the largest prime below n
    n -= 1 if n % 2 == 0 else 2 # Start with an odd number below n
    while (3 < n and not is_prime(n)): # While n is bigger than 3 or
        n -= 2                        # is not prime, go to next odd number
    return n                          # Return prime number or 3

```

The `doubleHashProbe()` generator starts by immediately yielding the first cell index folded over the possible range of cells. This is a little different from the approach in the linear and quadratic probes. The reason: you don't have to determine the step size if the first cell ends up being the desired one. Like the other probes, it doesn't need to know whether the caller seeks an empty or filled cell. If the caller finds an acceptable cell, it will exit its loop through the generator sequence, skipping the calculation of the step size. If the loop continues, then the generator calls `doubleHashStep()` to compute the step size. That value is used in a loop similar to those of the linear and quadratic probes. The difference is that it starts with 1 times the step size added to the start index and continues through all possible cells.

The `doubleHashStep()` function computes the step size for the given key. First, it gets the largest prime number that is smaller than the array size by calling `primeBelow()`. Next, it reapplies the hash function, `simpleHash()`, to the key (although it would be better to use a different hash function here). The large integer it produces gets mapped to the range of the smaller prime number using the modulo operator. The remainder is subtracted from the `prime` so that the step size falls in the range `[1, prime]`.

The `primeBelow()` function is a straightforward math calculation. It starts by finding an odd number below the parameter, `n`, by subtracting either 1 or 2 from it, depending on whether `n` is even or odd. The `while` loop decrements `n` by 2 until it either finds a prime or reaches 3.

To save some time, the `primeBelow` result could be stored in the `HashTable` object and recomputed only when the array size changes. The step size changes

based on the key, so it cannot be stored as efficiently. You would essentially need a hash table to look up the step size for each key!

Using Double Hashing in the Open Addressing Visualization Tool

As with the other methods, you can set the probe type whenever the hash table is empty in the HashTableOpenAddressing Visualization tool. To see a good example of the probes at work, you need to fill the table rather full, say to about nine-tenths capacity or more. Try creating a hash table of 41 cells with the maximum load factor of 0.9. Set the probe type by selecting the Use doubleHashProbe button. Fill most of the table with 30 random keys (perhaps without animating the hashing to go faster).

With such high load factors, only about a quarter of new, random data items will be inserted at the cell specified by the first hash function; most will require extended probe sequences. Try inserting one or two more random keys by using the Random Fill button with animation of the hashing.

Try finding some existing keys in the crowded table. When a search needs a multistep probe sequence, you'll see how all the steps are the same size for a given key, but that the step size is different. Some step sizes can be large and, when wrapping around the table size, can take what looks like a random path among the cells.

The visualization tool does not show the code for calculating the step size. In fact, the code is not shown for any of the probe sequence generators. You can still see the patterns they produce, however, by following the arrows indicating the cells being probed. The next section discusses the step-by-step execution of double hashing probes.

Double Hashing Example

The double hashing algorithm has many steps. If you haven't looked at the visualization tool, here's an example of how a series of insertions works. We start with an empty `HashTable` created with the `doubleHashProbe()` as its probe sequence (see [Listing 11-10](#)). We create the hash table with an initial size of 7 (which is different than visualization tool default size of 2). As we insert keys, the hash function computes where to store them and how big the steps should be to probe for open addresses. When the load factor gets too large, the table grows to accommodate more key-value pairs.

Table 11-2 shows how each insertion probes to find the cell to modify. The first key inserted is 1. The `simpleHash()` function just returns the same integer. That makes it easy to see how insertion works (but it's a terrible hash function in general). Using the same integer also clarifies what the step size will be for double hashing. The first prime below 7 is 5. Subtracting 1 mod 5 from 5 leaves 4 as the step size. Because cell 1 is empty, however, the step size doesn't matter, and item 1 is inserted into cell 1.

Table 11-2 *Filling a Hash Table Using Double Hashing*

Item Number	Key	simpleHash Value	Step Size	Total Cells	Prime Below	Probe Sequence After simpleHash
1	1	1	4	7	5	
2	38	3	2	7	5	
3	37	2	3	7	5	
4	16	2	4	7	5	6
5	20	3	6	17	13	9
6	3	3	10	17	13	13
7	11	11	2	17	13	
8	24	7	2	17	13	
9	4	4	9	17	13	13, 5
10	16*	16	15	37	31	
11	10	10	21	37	31	
12	31	31	31	37	31	
13	18	18	13	37	31	
14	12	12	19	37	31	
15	30	30	1	37	31	
16	1*	1	30	37	31	
17	19	19	12	37	31	
18	85	11	8	37	31	19, 27

The second key inserted, 38, follows the same pattern. After hashing and taking the modulo with 7, the probe starts at index 3. It would get a different step size of 2 based on $5 - (38 \bmod 5)$, but again, that doesn't matter because

cell 3 is empty. The third key inserted, 37, maps to index 2, which is also empty.

On the fourth key, 16, we hit the first collision. The `simpleHash()` maps it to index 2, which holds key 37. The step size for that key is $5 - (16 \bmod 5) = 4$. The second probe goes to cell 6 ($2 + 4$), which is empty, so key 16 gets stored there.

At this point, four items are stored in the hash table. The table has seven cells, so the load factor is now $4/7$, which is larger than the default `maxLoadFactor` of 0.5. The `insert()` method calls the `__growTable()` method after the fourth item is inserted. The table grows to hold 17 cells, and the four items are rehashed into them as follows:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	1		37	38												16

All of the keys rehash into cells of the expanded table at their initial hash addresses based on the new size.

The fifth item to insert has key 20. That hashes to cell 3 in the 17-cell table, which is occupied. The step size is 6, which is calculated using the largest prime below the table size which is 13 ($\text{step} = 13 - (20 \bmod 13)$). Probing six cells away finds cell nine to be empty, so that's where item 20 lands.

The sixth item to insert has key 3. That also hashes to the occupied cell 3. The secondary hashing leads to a step size of 10, and the next probe finds cell 13 to be empty. Item 3 gets placed in cell 13, avoiding the enlargement of any of the clusters. These last insertions illustrate how two keys, whose primary hash addresses collide, avoid creating clusters by using different step sizes after the initial probe.

The seventh item with key 11 finds cell 11 empty and lands there. That pattern repeats for item 8, key 24, which is stored in the empty cell 7.

More collisions occur with the ninth item, which has a key of 4. The contents of the array just before the insertion of that item are

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	1		37	38			24		20		11		3			16

Cell 4 is occupied, so it computes the step size of 9 ($13 - (4 \bmod 13)$). The next cell checked is 13. That cell is full, too, so it probes at 5 ($13 + 9 \% 17 = 5$), which is empty. Key 4 is stored there, forming a cluster of three filled cells.

Storing the ninth item in a table of 17 cells brings the load factor to $9/17$, which exceeds 0.5. The table grows again to hold 37 cells, and the nine items are reinserted into it. One collision occurs during the reinsertions (key 38 would normally go at index 1 in a 37-cell table but gets stored at cell 25 ($1 + (31 - (38 \bmod 31)))$).

As [Table 11-2](#) shows, items 10 through 17 are inserted without any collisions. That behavior is typical when the load factor is reduced after growing the table. Looking at the details, you can see that item 10 is actually a duplicate key to item 4, as indicated by the asterisk (*). Both have a key of 16, so the second insertion becomes an update to the value associated with that key. Item 16 is another duplicate; this time it's key 1. It also immediately finds the key at cell 1 and doesn't need to probe other cells before updating the value.

The eighteenth item produces a collision. It's a new key, 85, which hashes to cell 11 (because the table is now of length 37). Because key 11 is stored in cell 11, the double hashing determines that it should step by 8 cells. Probing cell 19 finds it occupied with key 19 from the previous insertion, so it moves on to store that key in the available cell 27.

As you can see from this example, collisions do cause some items to be placed in cells different than their original hash location. The chance of a collision increases as the load factor goes up. The 18 insertions filled 16 cells of the table because two of the keys were duplicates of previous keys. That puts the table load factor at $16/37$. After inserting three new keys, the table must grow again, keeping the chance of collision low and the number of clusters small.

Table Size a Prime Number

Double hashing requires the size of the hash table to be a prime number. To see why, imagine a situation in which the table size is not a prime number. For example, suppose the array size is 15 (indices from 0 to 14), and that a particular key hashes to an initial index of 0 and a step size of 5. The probe sequence would be 0, 5, 10, 0, 5, 10, and so on, repeating endlessly. Only these three cells would ever be examined, so the algorithm will never find the empty cells that might be waiting at 1, 2, 3, and so on. The reduced coverage of the

available cells means the algorithm will exhaust all its probes before quitting. In other words, it will crash and burn.

If the array size were instead 13, which is prime, the probe sequence would eventually visit every cell. It would be 0, 5, 10, 2, 7, 12, 4, 9, 1, 6, 11, 3, and so on. If there were even one empty cell, the probe would find it. Using a prime number as the array size makes it impossible for any number to divide it evenly (other than 1 and the prime itself), so the probe sequence will eventually check every cell.

Despite the added time needed to find prime numbers, double hashing wins overall when choosing the best probe sequence for open addressing.

Separate Chaining

In open addressing, collisions are resolved by looking for an open cell in the hash table. A different approach is to install a linked list or binary tree at each index in the hash table. A data item's key is hashed to the index in the usual way, and the item is inserted into the structure at that index. Other items that hash to the same index are simply added to the structure; there's no need to search for empty cells in the primary array. [Figure 11-17](#) shows how separate chaining looks. The top version shows sorted linked lists in the table cells, and the bottom shows balanced, binary trees.

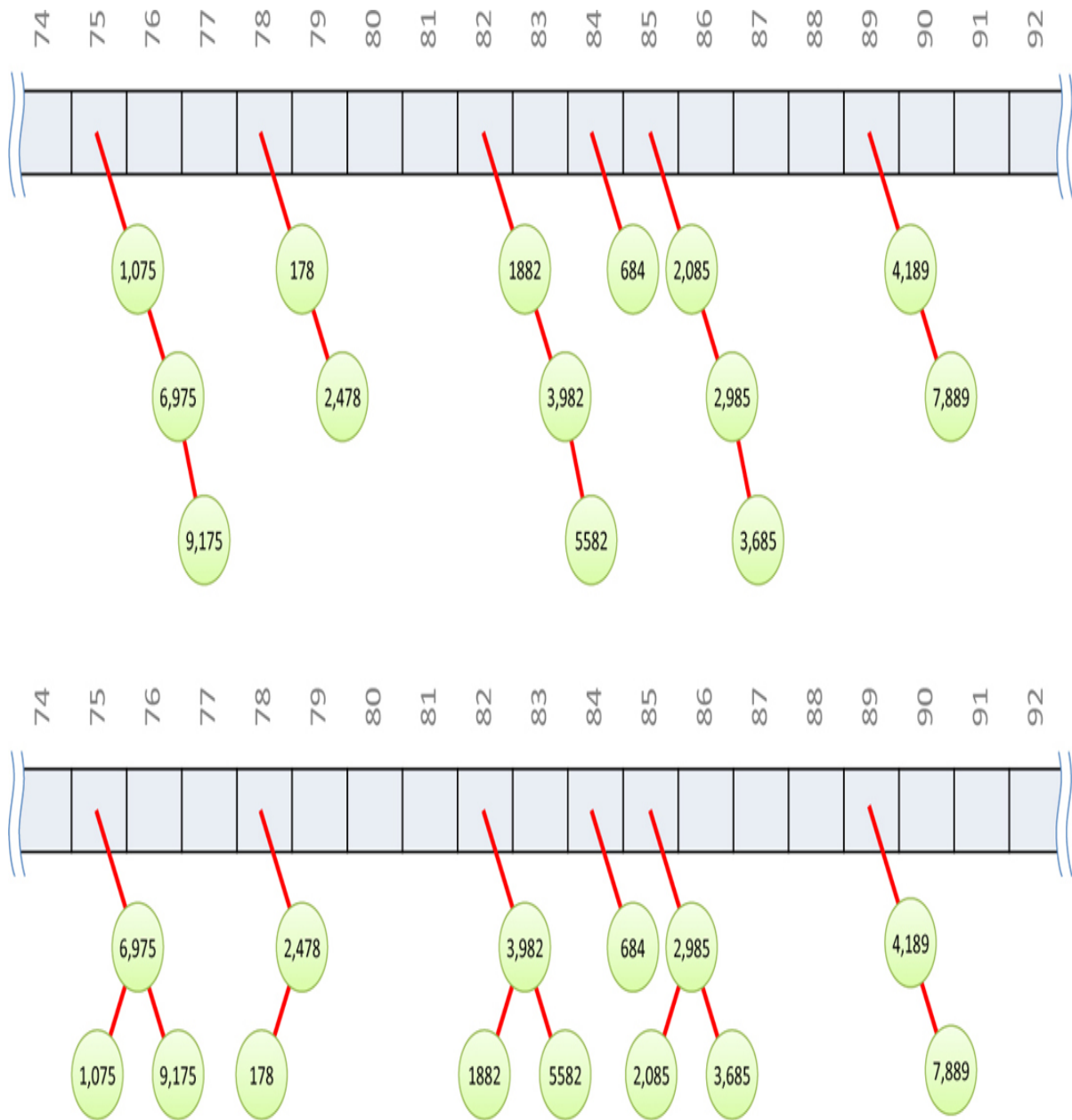


Figure 11-17 *Examples of separate chaining*

Separate chaining is conceptually somewhat simpler than the various probe schemes used in open addressing. The code, however, is longer because it must include the mechanism for the linked list or trees, usually in the form of an additional class.

The HashTableChaining Visualization Tool

To see how separate chaining works, start the HashTableChaining Visualization tool. It displays an empty array, as shown in [Figure 11-18](#). Like the open addressing tool, it starts with two empty cells that will grow as new items are added. The linked lists start from these cells and grow downward.



Figure 11-18 *Separate chaining in the HashTableChaining Visualization tool*

The buttons in the HashTableChaining Visualization tool are nearly identical to those of the HashTableOpenAddressing tool, except that the probe choice buttons have been removed. Despite the similar appearance, the operations have quite a few differences.

Try inserting some items. As with the open addressing tool, the keys can be strings or integers, and the same `simpleHash()` function determines which cell of the array should contain the key. The hashing box, item count, and load factor limit are at the top to keep them out of the way as the linked lists grow downward.

The linked lists are unsorted (we discuss the option of keeping them sorted shortly). New items are appended to the end of the list. After the total number of items in all the lists divided by the number of cells exceeds the `maxLoadFactor` limit, a similar `__growTable()` function creates an array that's at least twice as big with a prime number of cells. The items are rehashed to find their location in the new array.

It might seem strange that the separate chaining tool starts with a `maxLoadFactor` of 1.0 (100%). You saw how congested hash tables caused

problems in open addressing. Congestion can be a problem in separate chaining, too, but in a different way.

Try filling the array with 30 random keys (you can turn off the animation of the insertion by deselecting the Animate Hashing option before filling). You will likely get a table that looks similar to the one in [Figure 11-19](#). Most (14) of the linked lists have one item, a few (5) have two items, and two have three items in the example (the lists in cells 26 and 43). The 30 items are stored in 21 cells of the 47-cell table. That's typical of separate chaining; the items are spread out over many, but not all, of the cells and the length of the chains is relatively short.



Figure 11-19 *A separate chaining hash table with 30 random items*

With items in the hash table, try a few searches. If you click the index number for a cell, it will enter that number in the text entry box. You can use that index as a search key. If you click index 26 of the hash table in [Figure 11-19](#), enable the Animate Hashing option, and then select Search, you can see how the hashing finds cell 26 immediately and must step through each of the three items in the linked list to determine that no item with key 26 is in the hash table. Searches for existing keys also may step through a few items before finding the goal key.

Try deleting a few items (remembering that you can click a key to copy it to the text entry box). The search mechanism for the item to delete is the same, and the animation shows the steps it takes. Unlike open addressing, the item is deleted from the linked list, not replaced with a special deleted value.

In the visualization tool, the items are placed roughly below the hash table cell in which they belong but are adjusted in position to avoid obscuring other items and to reduce some of the overlapping arrows. The arrows have different lengths, making the number of links in each list harder to see. For example, cell 16 in [Figure 11-19](#) holds a linked list of one item, “AL-hlRfS,” and that item was placed below all the others due to the presence of other items inserted before it.

Load Factors

The load factor is typically managed differently in separate chaining than in open addressing. In separate chaining it's normal to put N or more items into an N cell array; thus, the load factor can be 1 or greater. There's no problem with this; some locations simply contain two or more items in their lists.

Of course, if the lists have many items, access time takes longer because access to a specified item requires searching through an average of half the items on the list. Finding the initial cell is fast, taking $O(1)$ time, but searching through a list takes time proportional to M , the average number of items on the list. This is $O(M)$ time. Thus, you don't want the lists to become too full. If you use binary trees, the search time is $O(\log M)$. You can let the trees grow more than lists do, if they remain balanced.

A load factor of 1, as shown in the initial visualization tool, is common. In open addressing, performance degrades badly as the load factor increases above one-half or two-thirds. In separate chaining the load factor can rise above 1 without hurting performance very much. This insensitivity to the load makes separate chaining a more robust mechanism and reduces unused memory, especially when it's hard to predict in advance how much data will be placed in the hash table. There is still a need to grow the hash table when the load factor (or the longest list or deepest tree) grows too big, but the decision metric is different.

You can experiment with load factor limits from 0.5 to almost 2.0 by creating new tables with the New button in the visualization tool. You can also fill with up to 99 randomly keyed items, and the growth of the hash table size is limited to 61 cells. This limitation creates some very congested separate chaining hash tables. The scroll bars will be adjusted to allow you to view the whole structure. The visualization slows down for large tables as it attempts to adjust the links to avoid overlaps.

Duplicates

Duplicate keys could be allowed in separate chaining but typically are not for all the same reasons applied to open addressing, plus the reasons mentioned for linked lists and binary trees. If they are allowed, all items with the same key will be inserted in the same list or tree. Therefore, if you need to discover all of them, you must search more of the list or tree in both successful and unsuccessful searches. Deletion must also search further when deleting all items with the same key. These extended searches lower performance.

Deletion

In separate chaining, deletion is easier than in open addressing. There's no need for a marker for a deleted item. The algorithm hashes to the proper table cell and then deletes the item from the list or tree. If the item is the last item stored in that cell, the cell can be set back to empty, if minimum memory usage is important. Leaving an empty tree or list header in the cell does not harm garbage collection.

Table Size

With separate chaining, making the table size a prime number is not as important as it is with open addressing. There are no probe sequences in separate chaining, so you don't need to worry that a probe will go into an endless sequence or not cover all the cells because the step size divides evenly into the array size.

On the other hand, certain kinds of key distributions can cause data to cluster when the array size is not a prime number. We have more to say about this problem when we discuss hash functions.

Buckets

Another approach similar to separate chaining is to use an array at each cell in the hash table instead of a linked list. Such fixed-size arrays are sometimes called **buckets** (although some hash table descriptions use the term "*bucket*" to mean what we've been describing as a "*cell*" of the hash table) This approach is not as efficient as the linked list approach, however, because of the problem of choosing the size of the buckets. If they're too small, they may overflow, requiring growth of the hash table and rehashing all the items. If the buckets

are too large, they waste memory. Linked lists and binary trees, which allocate memory dynamically, don't have this problem.

Python Code for Separate Chaining

There are many commonalities between the code for open addressing and separate chaining. [Listing 11-11](#) shows the core class definition for a separate chaining `HashTable`. Compare that with the definition shown in [Listing 11-2](#) for open addressing. There are three differences: there's an `import` statement to get the `KeyValueList` class, there's no `probe` parameter in the constructor because there is no probe sequence, and the default value for `maxLoadFactor` is higher. This class uses a slightly revised version of the `LinkedList` class from [Chapter 5](#), “[Linked Lists](#),” for the separate chaining. You could also use the `AVLTree` from [Chapter 10](#), “[AVL and Red-Black Trees](#).” It only needs to support the same interface of creation, insertion, search, deletion, and traversal. We discuss the `KeyValueList` class and the tree alternative later.

Listing 11-11 *The Core HashTable Class for Separate Chaining*

```
from KeyValueList import *

class HashTable(object):    # A hash table using separate chaining
    def __init__(           # The constructor takes the initial
        self, size=7,      # size of the table,
        hash=simpleHash,   # a hashing function, and
        maxLoadFactor=1.0): # the max load factor before growing
        self.__table = [None] * size # Allocate empty hash table
        self.__nItems = 0           # Track the count of items in the table
        self.__hash = hash          # Store given hash function, and max
        self.__maxLoadFactor = maxLoadFactor # load factor

    def __len__(self):       # The length of the hash table is the
        return self.__nItems # number of cells that have items

    def cells(self):         # Get the size of the hash table in
        return len(self.__table) # terms of the number of cells

    def hash(self, key):     # Use the hashing function to get the
        return self.__hash(key) % self.cells() # default cell index
```

The constructor for the class could initialize all the cells of the hash table to be empty linked lists. Doing so, however, would increase the memory consumed by the structure and add time to the construction process. It's preferable to wait until items need to be inserted in cells before creating the chain (list or tree) to hold them. Note that the utility methods `__len__()`, `cells()`, and `hash()` remain identical. Now let's look at what changes for separate chaining.

Perhaps the most significant change is the lack of a `__find()` method. For other data structures you used a find method to locate where an item is stored. In the case of separate chaining, you can use the hash function to find the cell (or bucket), and the chain's search and find methods to locate the item within that cell.

The `search()` method for the separate chaining `HashTable` simply hashes the key to a cell index, `i`; checks whether cell `i` has been filled with an list object; returns `None` if it doesn't or the result of searching for the key in the list. [Listing 11-12](#) shows the implementation. All the complexity of the probe sequence has gone away and is replaced by the, hopefully well-understood, operation of the chain structure (linked list or tree).

Listing 11-12 *The Search and Insert Methods for a Separate Chaining*

HashTable

```
class HashTable(object):    # A hash table using separate chaining
...
    def search(self,        # Get the value associated with a key
                key):       # in the hash table, if any
        i = self.hash(key)  # Get cell index by hashing key
        return (None if self.__table[i] is None else # If list exists,
                self.__table[i].search(key)) # search it, else None

    def insert(self,        # Insert or update the value associated
                key, value): # with a given key
        i = self.hash(key)  # Get cell index by hashing key
        if self.__table[i] is None: # If the cell is empty,
            self.__table[i] = KeyValueCollection() # Create empty linked list
        flag = self.__table[i].insert(key, value) # Insert item in list
        if flag:            # If a node was added,
            self.__nItems += 1 # increment item count
            if self.loadFactor() > self.__maxLoadFactor: # When load
                self.__growTable() # factor exceeds limit, grow table
        return flag         # Return flag to indicate update
```

```

def __growTable(self):    # Grow the table to accommodate more items
    oldTable = self.__table # Save old table
    size = len(oldTable) * 2 + 1 # Make new table at least 2 times
    while not is_prime(size): # bigger and a prime number of cells
        size += 2            # Only consider odd sizes
    self.__table = [None] * size # Allocate new table
    self.__nItems = 0          # Note that it is empty
    for i in range(len(oldTable)): # Loop through old cells and
        if oldTable[i]:          # if they contain a list, loop over
            for item in oldTable[i].traverse(): # all items
                self.insert(*item) # Re-hash the (key, value) tuple

```

The `insert()` method is also simplified. After hashing the key to get the cell index, `i`, it checks whether the cell is filled with a list object. If not, a new, empty key-value list is created and stored there. The main work of the insertion happens using the `KeyValueList`'s `insert()` method, shown later. Like the tree implementations, the `insert` method returns a flag indicating whether a new node was created (as opposed to updating an existing key's value). When a new node is created, it increments the number of items and checks the load factor. If it has crossed the threshold, the hash table must be enlarged. It is important to count the number of items in all the lists stored in hash table cells, not the number of lists, as we discuss shortly. After managing the growth of the table, `insert()` can return the `flag` indicating whether a new node was added to the hash table.

The `__growTable()` method starts off the same as for open addressing. It keeps a local variable pointing at the old table while it allocates a new one with a size that's a prime number at least twice the size of the previous one. Then it rehashes each of the items in the old table into the new one. The traversal of the old table is a loop through all its cells. Filled cells must be traversed using the chain's `traverse()` method. The key-value tuples are stored in the `item` variable and then passed as the two arguments to the hash table's own `insert()` method.

Let's look at the implementation of the `KeyValueList` class. It's a specialized version of the `LinkedList` class from [Chapter 5](#) where every link item holds a (key, value) tuple, as shown in [Listing 11-13](#). The definition starts off by importing the `LinkedList` class and defining some accessor functions for getting keys and values from the links.

Listing 11-13 *The Definition of the `KeyValueList` Class*

```
import LinkedList

def itemKey(item): return item[0] # Key is first element of item
def itemValue(item): return item[1] # Value is second element of item

class KeyValueList(LinkedList.LinkedList): # Customize LinkedList

    def insert(self, key, value): # Insert a key + value in list
        link = self.find(key, itemKey) # Find matching Link object
        if link is None: # If not found,
            super().insert((key, value)) # insert item at front
            return True # return success
        link.setData((key, value)) # Otherwise, update existing link's
        return False # datum and return no-insert flag

    def search(self, key): # Search by matching item key
        item = super().search(key, key=itemKey) # Locate key + value
        return itemValue(item) if item else None # Return value if any

    def delete(self, key): # Delete a key from the list
        try: # Try the LinkedList deletion by key
            item = super().delete(key, itemKey)
            return item # If no exceptions, return deleted item
        except: # All exceptions mean key was not
            return False # found, so return False

    def traverse(self): # Linked list traverse generator
        link = self.getFirst() # Start with first link
        while link is not None: # Keep going until no more links
            yield link.getData() # Yield the item
            link = link.getNext() # Move on to next link
```

The `insert()` method differs from its parent class in the way it handles insertion of duplicate keys. The simple `LinkedList` version always inserts new items at the beginning of the list. The `KeyValueList` must update an existing value if the key is already in the list. It first finds any `link` with a matching key (using the parent class's `find()` method with the `itemKey` function to extract the key from each link's tuple). If no such a link is found, it uses the parent class's `insert()` method to put the (key, value) tuple at the start of the list. It returns `True` to indicate the addition of a new item. If a link with a matching key is found, it updates the data for that link with the (key, value) tuple and returns `False` to indicate no additions were made.

The `search()` method uses the parent class's `search()` method to get the first link with a matching key, if any. It returns the associated value if a link is found and `None` otherwise.

The `delete()` method differs the most from its parent. In the simple `LinkedList` version, `delete()` throws an error if the list is empty or the key is not found. The `KeyValueList` uses a `try except` clause to get the result from the parent `delete()` method. If an item is found and deleted, it returns that item. If an error occurs, it returns `False` to indicate the key was not found.

Finally, the `traverse()` method of `KeyValueList` is nearly the same code as its parent class but uses a `yield` statement to make it a generator that yields the (key, value) tuples in the list.

Returning to the separate chaining `HashTable` implementation, you can define its `traverse()` method, as shown in [Listing 11-14](#), to use that of the `KeyValueList`. The loops are similar to those used in `__growTable()`. The differences are that the `traverse()` method loops over the current hash table cells instead of the old copy, and it yields the items it finds instead of reinserting them.

Listing 11-14 *The `traverse()` and `delete()` Methods for Separate Chaining*

```
class HashTable(object):    # A hash table using separate chaining
...
    def traverse(self):      # Traverse the key, value pairs in table
        for i in range(len(self.__table)): # Loop through all cells
            if self.__table[i]: # For those cells containing trees,
                for item in self.__table[i].traverse(): # traverse
                    yield item # the tree in-order yielding items

    def delete(self,        # Delete an item identified by its key
                key,        # from the hash table. Raise an exception
                ignoreMissing=False): # if not ignoring missing keys
        i = self.hash(key)    # Get cell index by hashing key
        if self.__table[i] is not None: # If cell i is not empty, try
            if self.__table[i].delete(key): # deleting item in tree and
                self.__nItems -= 1 # if found, reduce count of items
            return True        # Return flag showing item was deleted
        if ignoreMissing:     # Otherwise, no deletion. If we ignore
            return False      # missing items, return flag
```

```
raise Exception(           # Otherwise raise an exception
    'Cannot delete key {} not found in hash table'.format(key))
```

Before we discuss the `delete()` method, let's look at the order in which hash table items will be traversed.

Traversal Order in Hash Tables

What order should a hash table traverse its items? In binary search trees, you have the option to traverse in-order, pre-order, or post-order. Is there a way to do the same in hash tables? The orderings in trees are based on the structure of the tree with parent, left child, and right child nodes. There is no equivalent structure over all the items in the hash table (even though there can be such a structure in a single cell with separate chaining).

The order that the `traverse()` methods shown in [Listing 11-14](#) and [Listing 11-8](#) would yield items is based primarily on their hash address. That's a combination of both the hash function and the size of the hash table. If you wanted the keys to be yielded in ascending order, you would need to either reverse the hash function or collect all the keys and sort them. Reversing a hash function is very hard to do in most cases. Sorting the keys, as you've seen, could take $O(N \times \log N)$ time. In general, hash tables return the keys in "arbitrary" (unpredictable) order. If the caller needs them in order, it can sort them (by key or value). Interestingly, Python's `dict` hash table returned keys in arbitrary order in early versions. In version 3.7 and beyond, it returns them in insertion order.

Note that in separate chaining, the items within each cell are traversed according to the traversal order of the list or tree that is used. In the `KeyValueType` implementation, the list keeps the items in reverse insertion order. When the hash table grows, however, the rehashing of the items goes through the linked lists and reverses the previous insertion order. That makes the ultimate traversal order very hard to predict. If you use an AVL tree for chaining, then the items in each tree could be traversed in key order, but due to rehashing, they still will be yielded by the hash table's traversal in arbitrary order.

The `delete()` Method for Separate Chaining

The last method of the separately chained hash table implementation shown in [Listing 11-14](#) is `delete()`. Like `insert()`, the code for `delete()` is simpler

than what was needed for open addressing. This method computes the hash address of the key and then uses the `KeyValueList delete()` method to remove the item, if the list is present. If that list reports that an item was deleted, it reduces the item count for the hash table and returns `True` to signal the deletion of the item. If minimal memory usage is important, the cell should be set to `None` before returning `True` when the deletion results in an empty list or tree.

If the cell is empty or the tree doesn't find the `key` to delete, the `ignoreMissing` parameter determines whether to return `False` or raise an exception.

Which Type of Chaining Should You Use?

Separate chaining can use a number of secondary structures to store the items in each cell. Lists and trees are very common, and sometimes small arrays (when dynamic allocation of new storage is being avoided). There is no single type that's best for all use cases. Some structures are more efficient than others depending on the hash function and the keys inserted.

The most important factors are the maximum number of items in a cell and the number of times they will be searched. If the hash function is very good, it will distribute the keys evenly among all the table cells. If N items are stored in M table cells, the average number of items per cell is N/M . Note that this is exactly the same as the load factor for the hash table. In the separate chaining implementation, we used a `maxLoadFactor` of 1.0 as the default value. With a good hashing function, the average number of items stored in a cell should be 1, at most. There will be some cells with two items, some with none, and very few with three or more items.

If no cell contains more than three items, it makes sense to use the simplest of structures for separate chaining, the linked list. The items don't need to be kept sorted because you only need to compare at most three keys. Inserting into an unsorted list is normally an $O(1)$ operation. In the case of separate chaining, however, you must search the entire list to see if it is a duplicate key. Searching that list takes $O(N)$ —or $O(N/M)$ in this case—for the expected average length of the list. Maintaining the load factor, N/M , at 1.0 or below means that searching the unsorted list and inserting are both expected to be $O(1)$.

Sorted lists don't speed up a successful search, but they do cut the time of an unsuccessful search in half. As soon as an item larger than the search key is reached, which on average is half the items in a list, the search can be declared

a failure. That would become important the longer the lists grow. Deletion times are also cut in half in sorted lists.

If many unsuccessful searches are anticipated, it may be worthwhile to use the slightly more complicated sorted list rather than an unsorted list. An unsorted list, however, wins when insertion speed is more important. An example of that might be the use of a hash table to store all the entries for a “pick 6” lottery. Each lottery participant picks a sequence of six numbers. The hash table is used to store each participant’s contact info, so the key is the sequence of numbers, and the data is a list of people who picked that sequence. There could be millions of these sequences, but because there will be only one search for the winning sequence chosen by the lottery managers, there will be very few searches of the hash table (both successful and unsuccessful). There’s little point to spending time during the insertion to sort the keys.

The choice of the secondary structure to use in separate chaining can be affected by the choice of hash function too. Although there are many good hash functions, there are also some bad ones. If a particular application either chooses a bad hash function or somehow runs across a group of keys that hash to just one or two addresses in the hash table using that function, the number of items in one cell can grow as large as N . In that unlikely case, a balanced binary tree like the AVL tree could be best. That makes the insert and search operations within each cell $O(\log N)$ instead of $O(N)$. This is a degenerate case where the hash function doesn’t spread the data over a broad number of cells, so the more efficient tree structures are an improvement over lists.

We return to the question of when to use separate chaining versus open addressing when we discuss hash table efficiency later in this chapter.

Hash Functions

In this section we explore the issue of what makes a good hash function and see how we can improve the approach to hashing strings mentioned at the beginning of this chapter.

Quick Computation

A good hash function is simple, so it can be computed quickly. The major advantage of hash tables is their speed. If computing the hash function is slow,

this speed will be degraded. A hash function with many iterations or levels of computation is not a good idea. Many are based on sophisticated math. If they involve a lot of multiplications and divisions, especially on a computing platform without hardware support for those kinds of operations, they could take quite a bit of time.

The purpose of a hash function is to take a range of key values and transform them into index values in such a way that the hash addresses are distributed randomly across all the indices of the hash table. Keys may be completely random or not so random.

Random Keys

A so-called **perfect hash function** maps every key into a different table location. This is rarely possible in practice. A special case happens when the keys are unusually well behaved and fall in a range small enough to be used directly as array indices. For example, a manufacturer gives numbers for each of the parts it creates. The numbers started at 1,000 and go up to the number of things they have ever produced, say 10,000 over the past 50 years. Because they were created to be unique and with no gaps, these could easily be used directly as array indices without hashing, by simply having a hash function that subtracts 1,000 from the part number. These are unusually well-behaved keys.

If you need to store only a few of these part numbers, say the hundred parts currently kept in inventory, then it's possible to create a perfect hashing function that maps them to unique indices in a smaller array. The perfect hashing function needs to map each of the hundred part numbers to a unique index. You saw the Huffman coding algorithm in [Chapter 8](#), “[Binary Trees](#),” which came up with a unique bit sequence for every letter used in a message. Similar techniques can be used to assign a unique index to every part number.

In most applications, however, it's impossible to forecast what keys will be inserted in the hash table. Without knowing the number and type of keys, it's impossible to build a perfect hashing function. So typically, you make assumptions. In this chapter we've assumed that the transformed keys were randomly distributed over a large numeric range. In this situation the hash function

```
index = key % arraySize
```

is satisfactory. It involves only one more mathematical operation, and if the keys are truly random, the resulting indices will be random too, and therefore well distributed. If the keys share some common divisor(s), you can reduce the chance that they cause collisions by choosing `arraySize` to be a prime number (and hoping that prime number is not the common divisor).

Nonrandom Keys

Data is often distributed nonrandomly. In fact, it's very rare to find truly randomly (mathematicians would call it uniformly) distributed data.

Let's consider some examples for keys: a timestamp key such as the milliseconds that have elapsed since a particular point in time and an IP address on the Internet. Typically, these kinds of keys are not uniformly distributed across all the possible values; they concentrate in ranges. The millisecond timestamps may be for events over a short duration, such as log messages on a computer server over the past week, or perhaps for some past events, such as the births of a group of people. Those births are all likely to be concentrated on dates in the past century, not uniformly spread out over tens of thousands of years. Even within the past century, the birth rate rises and falls, leaving an uneven distribution. For the IP addresses, it's rare to get a set of data that samples the addresses from all over the world. Typically, there will be many references to local IP addresses and smaller numbers sprinkled from whatever regions communicated with the computer or network appliance that collects the data.

Many keys that might be used have an internal structure. IP addresses are 32-bit or 128-bit numbers organized into four octets or eight 16-bit words. Various blocks of addresses are reserved for different purposes. Any particular set of keys is likely to have many of its keys from a few blocks, and none from most of them.

Part numbers for manufacturers typically have structure too. Let's look at an example of a system that uses car part numbers as keys and discuss how they can be hashed effectively. Perhaps these part numbers are of the form

033-400-03-94-05-0-535

This number might be interpreted as follows:

Digits 0–2: Supplier number (1 to 999, currently up to 70)

- Digits 3–5: Category code (100, 150, 200, 250, up to 850)
- Digits 6–7: Month of introduction (1 to 12)
- Digits 8–9: Year of introduction (00 to 99)
- Digits 10–11: Serial number (1 to 99, but never exceeds 100)
- Digit 12: Toxic risk flag (0 or 1)
- Digits 13–15: Checksum (sum of other fields, modulo 1000)

If you ignore the separating hyphens, the decimal key used for the preceding 16-digit part number would be 0,334,000,394,050,535. The keys for the parts are not randomly distributed over all possible numbers. The majority of numbers from 0 to 9,999,999,999,999,999 can't actually occur (for example, supplier numbers higher than 70, category codes that aren't multiples of 50, and months from 13 to 99). Also, the checksum is not independent of the other numbers. Some work should be done to these part numbers to help ensure that they form a range of more truly random numbers.

Don't Use Nondata

The key fields should be squeezed down until every bit counts. For example, the category codes in the car part numbers should be changed to run from 0 to 15 (corresponding to the values 100, 150, ..., 850 that appear there). The checksum should be removed from the calculation of the hash because it doesn't add any additional information; it's deliberately redundant. Various other bit-twiddling techniques are appropriate for compressing the various fields in the key into the unique values they represent.

The address in memory of the key or the record containing the key should never be used in the hash function. In other words, if the key is accessed through a reference pointer, don't use the pointer when computing the hash. Use only the part number or other identifying elements referenced by the pointer. The location in memory where the data is stored changes from run to run. Using that location would mean that keys would only match on some runs of the program.

Use All the Data

Every part of the key (except nondata, as just described) should contribute to the hash function. Don't just use the first four digits, last four digits, or some

such abbreviation. The more data that contributes to the key, the more likely it is that the keys will hash evenly into the entire range of indices.

Sometimes the range of keys is so large that it overflows the type of integer values that the programming language supports. Most computing platforms support 32-bit and 64-bit integers. Some embedded processors, however, might support only 16-bit or 8-bit. Regardless of the platform size limit, there will be keys with numeric values that go beyond what can be represented in a single machine word. We show how to handle that overflow when we talk about hashing strings in a moment.

To summarize: The trick is to find a hash function that's simple and fast, using all the available data, while excluding the nondata and redundant parts of the key.

Use a Prime Number for the Modulo Base

Often the hash function involves creating a number on a large range and using the modulo operator (%) with the table size to map it to the hash address. You've already seen that it's important for the table size to be a prime number when using a quadratic probe or double hashing. If the keys themselves are not randomly distributed, it's important for the table size to be a prime number no matter what hashing system is used.

To see why a prime number of cells is helpful, consider what happens if many hashed keys are separated in value by some number, X . If X is a divisor of the array size, say $\frac{1}{4}$ of the size, that large group of keys hash to the same 4 locations, causing primary clustering. Using a prime table size nearly eliminates this possibility. For example, if the table size were a multiple of 50 in the car part example, the category codes could all hash to index numbers that are multiples of 50 (assuming that code is multiplied into the hash value). With a prime number such as 53, however, you are guaranteed that only keys that hash to multiples of that prime (plus a constant offset) are hashed to the same address. Part numbering and other man-made schemes rarely use prime numbers like that.

Returning to the example of timestamps as keys, the events represented by timestamps are often periodic. The times can represent things that usually happen at the top of the hour, or every 20 minutes, or every year. The periodic events will create timestamps bunched together around certain peak times. If those peaks are separated by a multiple of the array size, then many keys hash

to the same address. Even when they don't land exactly on multiples of the array size, the bunches can create collisions of hash addresses causing clusters in open addressing or long chains in separate chaining.

The moral is to examine your keys carefully and tailor your hash algorithm to remove any regularities in the distribution of the keys.

Hashing Strings

At the beginning of this chapter, you saw how to convert short strings to key numbers by multiplying digit codes by powers of a constant. In particular, you saw that the three-letter word *elf* could turn into the number 3,975 by calculating

$$\text{key} = 5*27^2 + 12*27^1 + 6*27^0$$

This approach has the desirable attribute of involving all the characters in the input string. The calculated key value can then be hashed into an array index in the usual way:

```
index = key % arraySize
```

The `simpleHash()` method shown in [Listing 11-3](#) used a similar calculation but with a base of 256 instead of 27. This calculation allows for many more possible characters in the string (but not all Unicode values).

The `simpleHash()` method is not as efficient as it might be. When hashing strings, it performs the character conversion, raises 256 to the power *i* (the position of character in the string), multiplies them, and adds all products in the `sum` expression:

```
sum(256 ** i * ord(key[i]) for i in range(len(key)))
```

This way of expressing the calculation is concise, but it does some extra work that can be avoided. You can eliminate computing the power of 256 by taking advantage of a mathematical identity called Horner's method. This method states that an expression like

$$\text{var4} * n^4 + \text{var3} * n^3 + \text{var2} * n^2 + \text{var1} * n^1 + \text{var0} * n^0$$

can be written as

$$(((\text{var4} * n + \text{var3}) * n + \text{var2}) * n + \text{var1}) * n + \text{var0}$$

The base, `n`, now appears without an exponent, multiplying each parenthesized expression (including `var4`). To convert this equation into loop form, you start inside the innermost parentheses and work outward. Translating this equation to a Python function results in the following:

```
def hashString1(key):          # Use Horner's method to hash a string
    total = 0                  # Sum contribution of all characters
    for i in range(len(key) - 1, -1, -1): # Go in reverse order
        total = total * 256 + ord(key[i]) # Multiply by base, add char i
    return total                # Return sum
```

The `hashString1()` function computes the same hash that `simpleHash()` does for a string, but with one multiply and one addition per character in the loop. Raising 256 to a power happens by repeating the multiplications.

This approach is a definite improvement because most processors can perform multiplications in a few clock cycles, whereas raising numbers to a power can take much longer. There are two more changes that can help a little more. Multiplying by a power of 2 is the same as shifting the bits of a binary number to the left by that power. In this case, 256 is 2^8 , and you can use the bit shift operator, `<<`, instead of the multiplication:

```
def hashString2(key):          # Use Horner's method to hash a string
    total = 0                  # Sum contribution of all characters
    for i in range(len(key)):  # Go in forward order
        total = (total << 8) + ord(key[i]) # Shift to mult., add char i
    return total                # Return sum
```

Bit shifts are supported by every modern processor and are usually faster than multiplication. This example also changes the character index, `i`, to increase from 0 to the last character of the key. This approach saves a tiny bit of a time (a subtraction) and makes the code simpler, although it will produce very different hash values than `hashString1()` for the same string (other than palindromes). The strings still get hashed to unique values, but the most significant characters—the ones that change the hash value the most—are on the left for the `hashString2()` function.

The `hashString2()` function provides a more optimized hash function that will compute a unique number very quickly from every string. There's another factor to consider, however, and that is the magnitude of the sum. As you shift bits (or multiply), you eventually create numbers bigger than what fits in a machine word. A 64-bit processor could shift the `total` by 8 bits seven times

without overflowing its 64-bit registers. For strings longer than 8 characters (or character points greater than 255), an overflow is likely to occur.

Can we modify the basic approach so we don't overflow any variables? Notice that the hash address we eventually end up with is always less than the array size because we apply the modulo operator. It's not the final index that's too big; it's the intermediate `total` values.

With any arithmetic expression using `+`, `*`, and `-`, you can apply the modulo operator (`%`) at each step in the calculation. Using the operator this way gives the same result as applying the modulo operator once at the end but avoids overflow, at the cost of adding an operation inside the loop. The `hashString3()` method shows how this looks:

```
def hashString3(key, size): # Use Horner's method to hash a string
    total = 0                # Sum without overflowing
    for i in range(len(key)): # Go in forward order, shift, add char i
        total = ((total << 8) + ord(key[i])) % size # and use modulo
    return total              # Return sum
```

Most string hashing functions take this approach (or something like it). Various bit-manipulation tricks can be played as well, such as using a size that is a power of 2. That means the modulo operator can be replaced by AND-ing the total with a bit “mask” (for example, `total & 0xFFFFFFFF`). On the other hand, using a power of 2 as the hash table size means that there could be hash table collisions for keys with patterns related to that power of 2, rather than a prime.

You can use similar approaches to convert any kind of string or byte sequence to a number suitable for hashing. Because all data is stored as sequences of bytes, this scheme handles nearly any kind of data.

Folding

Another reasonable hash function involves breaking the key into groups of digits and adding the groups. This approach ensures that all the digits influence the hash value. The number of digits in a group should correspond to the size of the array. That is, for an array of 1,000 items, use groups of three digits each. The **folding** technique is almost like writing the digit string on a strip of paper, folding the paper between every group of `K` digits, and then adding the numbers now piled on top of one another.

For example, suppose you want to hash 10-digit telephone numbers for linear probing. If the array size is 1,000, you would divide the 10-digit number into three groups of 3 digits, plus a final digit. If a particular telephone number was 123-456-7890, you would calculate a key value of $123+456+789+0 = 1368$. The modulo operator can map those sums to the range of indices, 0–999. In this case, $1368 \% 1000 = 368$. If the array size is 100, you would need to break the 10-digit key into five 2-digit numbers: $12+34+56+78+90 = 270$, and $270 \% 100 = 70$.

It's easier to imagine how this operation works when the array size is a multiple of 10. For best results, however, the array size should be a prime number, or perhaps a power of 2, as you've seen for other hash functions. We leave an implementation of this scheme as an exercise.

Hashing Efficiency

We've noted that insertion and searching in hash tables can approach $O(1)$ time. If no collision occurs, or the separate chains contain one element at most, only a call to the hash function, an array reference, and maybe a link dereference are necessary to find an existing item or insert a new item. This is the minimum access time.

Note that the hash function takes some time to compute, and the amount of time depends on the length of the key and the hashing function. Keys are typically short, perhaps tens of bytes. Because the length of the keys is much shorter than a large N —the number of items stored—you treat the time spent hashing as $O(1)$. When you're considering hashing a large sequence of bytes—say an entire video file—the time spent hashing could become significant, but it still could be small compared the number of items stored (for example, all the videos available on the Internet).

If collisions occur, access times become dependent on the resulting probe lengths or search of a chain. Each cell accessed during a probe or link in a chain adds another time increment to the search for a vacant cell (for insertion) or for an existing cell. During an access, a cell or link must be checked to see whether it's empty and whether it contains the desired item.

Thus, an individual search or insertion time is proportional to the length of the probe, length of the chain, or depth of the tree. This variable time must be added to the constant time for the hash function.

The average probe or chain length (and therefore the average access time) is dependent on the load factor (the ratio of items in the table to the size of the table). As the load factor increases, the lengths grow longer.

Let's look at the relationship between probe lengths and load factors for the various kinds of hash tables we've studied.

Open Addressing

The loss of efficiency with high load factors is more serious for the various open addressing schemes than for separate chaining.

In open addressing, unsuccessful searches generally take longer than successful searches. Remember that during a probe sequence, the algorithm stops as soon as it finds the desired item, which is, on average, halfway through the probe sequence. On the other hand, probing must go all the way to the end of the sequence before it's sure it can't find an item.

Linear Probing

The following equations show the relationship between probe length (P) and load factor (L) for linear probing. For a successful search, it's

$$P = (1 + 1 / (1 - L)) / 2$$

and for an unsuccessful search, it's

$$P = (1 + 1 / (1 - L)^2) / 2$$

These formulas are from Knuth (see [Appendix B](#), “[Further Reading](#)”), and their derivation is quite complicated. [Figure 11-20](#) shows the graphs of these equations in the blue (upper) curves. The upper graph shows the probe lengths for successful searches and the lower one shows the lengths for unsuccessful searches.



Figure 11-20 *Successful and unsuccessful probe performance*

At a load factor of 0.5, the average successful search takes 1.5 comparisons, and the average unsuccessful search takes 2.5. At a load factor of 2/3, the numbers are 2.0 and 5.0. At higher load factors, the numbers become very large—so high they go off the graph to infinity. We discuss the other lines in the graphs shortly.

The takeaway, as you can see, is that the load factor must be kept under 2/3 and preferably under 1/2. On the other hand, the lower the load factor, the more memory is needed for a given number of items. The optimum load factor in a particular situation depends on the trade-off between memory efficiency, which decreases with lower load factors, and speed, which increases.

Quadratic Probing and Double Hashing

Quadratic probing and double hashing share their performance equations. These equations indicate a modest superiority over linear probing. For a successful search, the formula (again from Knuth) is

$$P = -\ln(1 - L) / L$$

where $\ln()$ is the natural logarithm function. This is like $\log_2()$, except the base is the special constant, $e \cong 2.718$. For an unsuccessful search, it is

$$P = 1 / (1 - L)$$

Figure 11-20 shows the graphs of these formulas using red lines. At a load factor of 0.5, successful searches take about 1.4 probes, whereas unsuccessful ones average 2.0. At a 2/3 load factor, the numbers are about 1.6 and 3.0; and

at 0.8, they're 2.0 and 5.0. Thus, somewhat higher load factors can be tolerated for quadratic probing and double hashing than for linear probing. That shows up in the graph as the red lines lying below the blue lines.

Note that both the red and blue lines climb steeply as the load factor approaches 1.0. That behavior is expected because it means the table is nearly full, and finding a key or an empty slot can take up to N probes.

Separate Chaining

The efficiency analysis for separate chaining is different, and generally easier, than for open addressing.

We want to know how long it takes to search for a key or to insert an item with a new key into a separate-chaining hash table. All of the methods must compute the hash function and determine a starting hash address. The time taken to compute that is a constant, so we focus on the number of key comparisons needed when searching the chain structure. For chaining, we assume that determining when the end of a list or tree has been reached is equivalent to one key comparison. Thus, all operations require $1 + n_{\text{Comps}}$ time, where n_{Comps} is the number of key comparisons.

Say that the hash table contains `size` cells, each of which holds a list, and that N data items have been inserted in the table. Then, on average, each list holds N divided by `size` items:

$$\text{Average List Length} = N / \text{size}$$

This is the same as the definition of the load factor, L:

$$L = N / \text{size}$$

Therefore, the average list length equals the load factor.

Searching

In a successful search, the algorithm hashes to the appropriate list and then searches along the linked list for the item. On average, half the items must be examined before the correct one is located. Thus, the search time is

$$P = 1 + L / 2$$

This is true whether the lists are ordered or not. In an unsuccessful search, if the lists are unordered, all the items must be searched, so the time is

$$P = 1 + L$$

These formulas are graphed in [Figure 11-20](#) using the green (lowest) lines. For an ordered list, only half the items must be examined in an unsuccessful search, so the time is the same as for a successful search.

In separate chaining it's typical to use a load factor of about 1.0 (the number of data items equals the array size). Smaller load factors don't improve performance significantly, but the time for all operations increases linearly with load factor, so going beyond 2 or so is generally a bad idea. Of course, the open addressing methods must keep the load factor well below 1.0.

Insertion

On the face of it, insertion of a new key is immediate, in the sense that no comparisons are necessary. Because we chose to not allow duplicate keys, however, any existing chain must be searched to determine whether the key is new or a duplicate. That means that insertion behaves exactly like search plus some constant work to either insert the new item at the end (or beginning) of the list or to update the existing data. The hash function must still be computed, and the data inserted or updated, so let's call the insertion time 1. To stay consistent with the other measures, you can call that a probe length, P , of 1. Finding that the key is new is equivalent to the time taken for an unsuccessful search of an unordered list:

$$P = 1 + L$$

If the lists are ordered or the key exists in the chain, then, as with an unsuccessful search, an average of half the items in each list must be examined, so the insertion time is

$$P = 1 + L / 2$$

Separate Chaining with Binary Search Trees

If binary search trees are used to organize the items in each cell, there are a few differences from separate chaining with lists. If you want to get the benefit of fast search of the binary tree, it makes the most sense to use one of the self-

balancing binary search tree structures (for example, AVL, 2-3-4, or red-black trees). They are more complex to code, but the number of comparisons needed in both successful and unsuccessful searches is proportional to the depth of the tree. The average number of items stored in each tree is the load factor, like it was for lists. That means the average depth of the trees is $\log_2(L)$. When the load factor is zero, there still is one probe, so the probe lengths are approximately

$$P = 1 + \log_2(L + 1)$$

Inserting into the tree requires finding where the new key belongs, which takes $1 + \log_2(L + 1)$ steps. Finding an existing key also takes $1 + \log_2(L + 1)$ steps. Searching for a key that is not in the tree stops when there is no child node where the key would normally fit, so it too takes the same number of steps. There are small variations between the exact number of steps in each of these cases.

Compared with the other methods shown in [Figure 11-20](#), the graph for binary search trees would be just below the green line for separate chaining. Both graphs would start at 1.0 and slowly rise as the load factor increases, but the binary search trees would rise more slowly after the load factor becomes greater than 1.0. The difference is so minor for low load factors that the simplicity of chaining with lists outweighs the probe performance. The faster search is a benefit, only if the load factor gets large, or a bad combination of the hash function with the keys being hashed puts large fractions of the N items in a single tree.

Growing Hash Tables

Along with the time spent probing for where to insert a new item, there is also the time spent growing the hash table and rehashing items already stored in it. Both open addressing and separate chaining benefit by keeping the load factor low, so they typically double the hash table size when the load factor exceeds a threshold (that differs for the two types).

How much extra work does reinserting the items cause? Consider the first insertions: when the load factor is low, insertion of a single item happens in $O(1)$ or “constant” time. If you never had to rehash the items, then inserting N of them takes $O(N)$ time. It may not seem intuitive, but allowing hash tables to grow exponentially by doubling in size maintains that $O(N)$ performance.

To see why, let's assume that you start with a table of size 1 and that you double it every time an insertion makes the load factor exceed 0.5 (we are intentionally setting aside the complexity of choosing prime table sizes here). After the first insertion, the table is doubled to two cells, and the one item is reinserted. The second insertion pushes the load factor over the threshold again, and the two items must be reinserted into the four-cell table. The steps are detailed in [Table 11-3](#).

Table 11-3 *Reinsertions When Table Size Doubles*

Item	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Table size	2	4	8	16	16	16	16	32	32	32	32	32	32	32	32	64
Reinsertions	1	2	3	4				8								16

Item	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
Table size	64	64	64	64	64	64	64	64	64	64	64	64	64	64	64	128
Reinsertions																32

At the insertion of the fourth item, the table doubles to hold 16 items, and the 4 items must be reinserted. The fifth item, however, does not cause any doubling or reinsertion. The same conditions hold until you get to the eighth item, when another doubling happens, and the 8 items must be reinserted.

As [Table 11-3](#) shows, the reinsertion work happens every time the number of items reaches another power of 2. There are longer and longer intervals between those expansions. The whole second section of [Table 11-3](#) has no reinsertions until item 32 is inserted. By the time you insert some large number of items, say a million, the number of reinsertions will be

$$1 + 2 + 3 + 4 + 8 + 16 + \dots + 262,144 + 524,288$$

The last number in the sum is the largest power of 2 below one million. Moving out the exception for 3 and writing these as powers of 2 makes the sum:

$$3 + 2^0 + 2^1 + 2^2 + 2^3 + 2^4 + \dots + 2^{18} + 2^{19}$$

The sum of the powers of 2 should look familiar. It is the same as the count of the nodes in a binary tree going from the root to the leaves. As you saw when

analyzing the efficiency of binary trees in [Chapter 8](#), that sum of powers of 2 going from 0 to some level, K, can be written as a formula that depends on a power of 2 itself:

$$2^0 + 2^1 + \dots + 2^K = \sum_{i=0}^K 2^i = 2^{K+1} - 1$$

So, the total number of reinsertions is $3 + 2^{K+1} - 1$ or $2 + 2^{K+1}$. That looks as though it could become a very large number, which means inserting N items would take a lot more work than $O(N)$. What you have to remember is the relationship between K and N. When you insert a million items, K is 19; and 2^K is the largest power of 2 less than or equal to N. In other words, K is the integer just below $\log_2(N)$. Substituting $\log_2(N)$ in the equation for the number of reinsertions (and forgetting about getting the integer just below it) leaves

$$\begin{aligned} \text{number of reinsertions} &= 2 + 2^{\log_2(N)+1} = 2 + 2[\text{ts}]2^{\log_2(N)} = 2 + 2[\text{ts}] N \\ &= 2(N + 1) \end{aligned}$$

That means the number of reinsertions grows linearly as N grows. The original N items were inserted with $O(N)$ work and the reinsertions just increase the constant multiplying N, so the overall complexity is still $O(N)$. Even when you grow the table to be the next prime number larger than twice its current size, the same pattern holds. The number of items to reinsert is always less than half the new table size, and the overall sum of reinserted items does not grow faster than $O(N)$.

Hash Table Memory Efficiency

We've noted that hash tables can contain many unused cells under various circumstances. Overall, they still consume $O(N)$ memory to store N items. They need more memory than a simple array because they must store keys along with their associated values (arrays store only the values, and the key is implicit). Keeping the load factor below the thresholds means you need roughly twice as many cells as the number of items, N, but that is still $O(N)$.

We have assumed that the table grows during insertions by doubling its size when needed so that it is never much more than twice N. When you look a bit closer, however, if you first insert P items and then delete some of them to reach N items, open addressing implementations will consume $O(P)$ space. That amount could be significant when P is much larger than N.

For separately chained hash tables, the process of inserting P items and then deleting some to leave N items doesn't waste quite as much memory as open addressing does because deletions within a chain or tree free up the memory that was being consumed. Only deleting the last item in the chain or tree leaves an empty memory cell or empty chain object.

The other item to note is that *traversal time is proportional to the table size*. If many items were deleted from a hash table that once held P items, traversal time would still take $O(P)$ to check all the cells. This is the first data structure we've seen where traversal could be a bit slower than $O(N)$ where N is the current number of items stored. Deletions in hash tables cause the difference in the time efficiency of traversal. It remains $O(N)$, but N is the maximum number of items inserted, not the current number stored.

Open Addressing Versus Separate Chaining

If open addressing is to be used, double hashing is the preferred system (certainly over quadratic probing). The exception is the situation in which plenty of memory is available and the data won't expand after the table is created; in this case, linear probing is somewhat simpler to implement and, if load factors below 0.5 are used, causes little performance penalty.

The number of items that will be inserted in a hash table generally isn't known when the data structure implementation is written, and sometimes not even when the table is created. Thus, in most cases, separate chaining is preferable to open addressing. Allowing the load factor to get large causes major performance penalties in open addressing, but performance degrades only linearly or logarithmically in separate chaining.

When in doubt, use separate chaining. Its drawback is the need for a linked list or binary search tree class, but the payoff is that adding more data than you anticipated won't cause performance to slow to a crawl.

Hashing and External Storage

At the end of [Chapter 9, "2-3-4 Trees and External Storage,"](#) we discussed using B-trees as data structures for external (disk-based) storage. Let's look briefly at the use of hash tables for external storage.

Recall from [Chapter 9](#) that a disk file is divided into blocks containing many records and that the time to access a block is much longer than any internal processing on data in main memory. For these reasons, the overriding consideration in devising an external storage strategy is minimizing the number of block accesses.

On the other hand, external storage is less expensive per byte, so it may be acceptable to use large amounts of it, more than is strictly required to hold the data, if by so doing you can speed up access time. Hash tables make this speed up possible.

Table of File Pointers

The central feature in external hashing is a hash table containing block numbers, which refer to blocks in external storage. This configuration is similar to the separately chained hash table, but the contents of the table cells point to external blocks rather than a list or a tree in memory. The hash table is sometimes called an **index** (in the sense of a book's index). It can be stored in main memory or, if it is too large, stored externally on disk, with only part of it being read into main memory at a time. Even if it fits entirely in main memory, a copy will probably be maintained on the disk and read into memory when the file is opened.

Nonfull Blocks

Let's assume the same characteristics as the contact database example from [Chapter 9](#) in which the block size is 8,192 bytes, and a record is 1,024 bytes. Thus, a block can hold 8 records. Every entry in the hash table points to one of these blocks. Let's say there are 100 blocks in a particular file.

The index (hash table) in main memory holds pointers to the file blocks. The hash table has indices from 0 to 99. The contents of cell 11, for example, holds a block number in external storage where a group of records are stored. Those records' keys hash to 11.

In external hashing it's important that blocks don't become full. Thus, you might store an average that's about half the maximum capacity, so 4 records per block in this example. Some blocks would have more records, and some fewer. There would be about 400 records in the file. This arrangement is shown in [Figure 11-21](#).

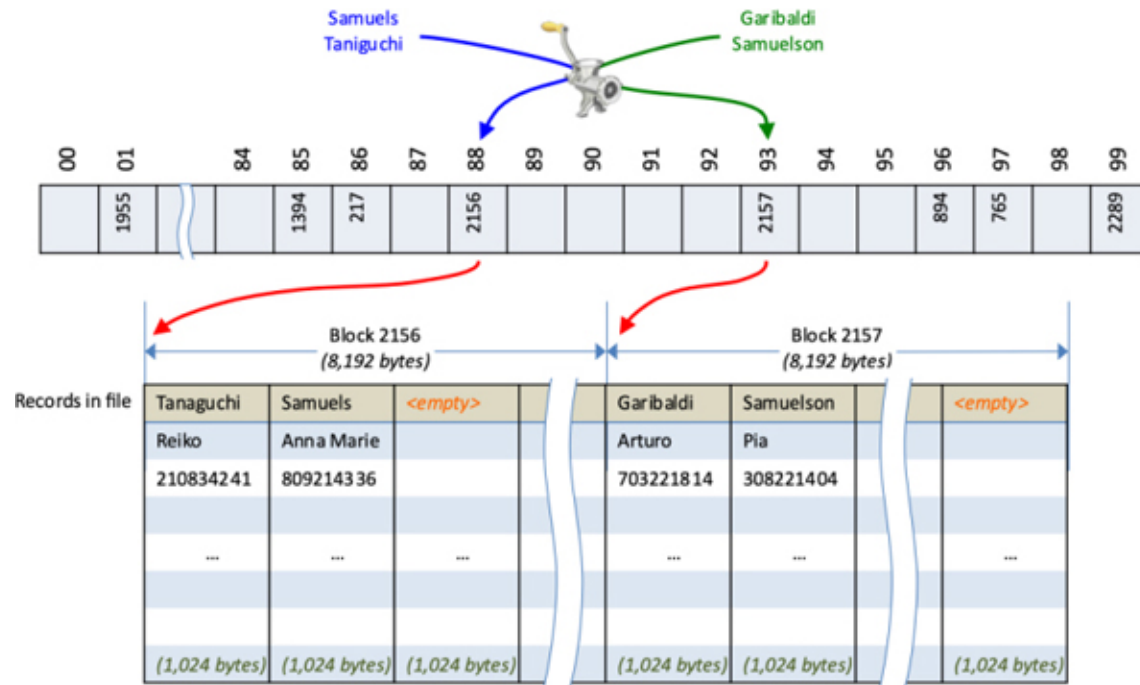


Figure 11-21 *Hashing to external blocks*

All records with keys that hash to the same value are located in the same block. To find a record with a particular key, the search algorithm hashes the key, uses the hash value as an index to the hash table, gets the block number at that index, and reads the block. In the [Figure 11-21](#) example, the name *Samuels* hashes to address 88 in the 100-block index. That contains block number 2156. Reading that block from disk allows the algorithm to retrieve the full record for Samuels. A similar name like *Samuelson* could hash to a very different address, 93 in the example, and be stored in another block. Different names like *Taniguchi* could hash to the same table address, and block, as *Samuels*. A new key might hash to an empty cell that has no block number.

This process is efficient because only one block access is necessary to locate a given item. The downside is that considerable disk space is wasted because the blocks are, by design, not full. The example in [Figure 11-21](#) shows two blocks, each holding only two records. For best performance, the load factor would need to be kept between 0.5 and 1.0.

To implement this scheme, you must choose the hash function and the size of the hash table with some care so that a limited number of keys hash to the same value. In this example, you want only four records per key, on the average.

Full Blocks

Even with a good hash function, a block will occasionally become full. This situation can be handled using variations of the collision-resolution schemes discussed for internal hash tables: open addressing and separate chaining.

In open addressing, if, during insertion, one block is found to be full, the algorithm inserts the new record in a neighboring block. In linear probing, this is the next block, but it could also be selected using double hashing. In separate chaining, special overflow blocks are made available; when a primary block is found to be full, the new record is inserted in the overflow block. Overflow blocks can be chained to allow large overflows, although these impact performance significantly.

Full blocks are undesirable because an additional disk access is necessary for the second and any subsequent overflow block(s); and the disk block access time can be many tens or thousands of times longer than a memory access. Such long access times may still be acceptable, however, if it happens rarely.

We've discussed only the simplest hash table implementation for external storage. One simple improvement on the example shown in [Figure 11-21](#) is to add a record index to the hash table in addition to the block number. That would allow constant time retrieval of the record within the block after it's read for only a few bytes more storage per cell. There are many more complex approaches that are beyond the scope of this book.

Summary

- A hash table is based on an array.
- The range of possible key values is usually greater than the size of the array.
- A key value is hashed to a number by a hash function.
- The hashed number is mapped to an array index called the hash address, typically using a modulo operation.
- An English-language dictionary—where words are the keys and definitions are the values—is a typical example of a database that can be efficiently handled with a hash table.